

UNIVERSIDAD NACIONAL DEL ALTIPLANO

Facultad de Ingeniería Mecánica Eléctrica,
Electrónica y Sistemas

Escuela Profesional de Ingeniería de Sistemas

PRÁCTICA SEMANA 1

POO II – SIS206

**Revisión de POO I: Diagnóstico,
Refactorización y Calidad de Código**

Estudiante: Kevin

Curso: Programación Orientada a Objetos II

Código: SIS206

Semestre: 2026-II

Docente: Mg. Aldo Hernán Zanabria Gálvez

Puno – Perú
2026

Índice

1	Introducción	2
2	Marco Teórico	2
2.1	Actividad 1 — Diagnóstico: Identificación de problemas en código existente (Python / C++)	2
2.2	Actividad 2 — Refactorización en Python: Misma lógica, mejor diseño (Python / C++)	3
2.3	Actividad 3 — Equivalente en C++17: Encapsulamiento y validación (Python / C++)	6
2.4	Actividad 4 — Cohesión y acoplamiento: Análisis comparativo (Python / C++)	9
3	Conclusiones	11
4	Referencias	12

1. Introducción

El diseño orientado a objetos no se limita a organizar código en clases y métodos: implica tomar decisiones que determinan qué tan fácil será mantener, extender y depurar un sistema en el futuro. Un código que "funciona" no es necesariamente un código bien diseñado, y es común encontrar programas que, aunque cumplen su propósito inmediato, presentan defectos de diseño como nombres poco descriptivos, falta de encapsulamiento, ausencia de validación de datos o clases con demasiadas responsabilidades.

La presente práctica correspondiente a la Semana 1 del curso de Programación Orientada a Objetos II tiene como propósito desarrollar la capacidad de identificar este tipo de defectos en código existente y aplicar correcciones basadas en buenas prácticas de la programación orientada a objetos (POO). Para ello, se parte de un código en Python con múltiples problemas de diseño, el cual es analizado, refactorizado y posteriormente reescrito en C++17, lenguaje que exige un manejo más estricto del encapsulamiento y del control de errores.

A lo largo del documento se desarrollan cuatro actividades: primero, un diagnóstico de los defectos presentes en el código original; segundo, una refactorización en Python que aplica nombres descriptivos, atributos privados, propiedades y validación de datos; tercero, la implementación equivalente en C++17 con encapsulamiento estricto y manejo de excepciones; y cuarto, un análisis comparativo entre dos diseños distintos para evaluar conceptos de cohesión, acoplamiento y el Principio de Responsabilidad Única (SRP). Con este recorrido se busca reforzar, de manera práctica, los fundamentos que sustentan un diseño de software robusto y mantenible.

2. Marco Teórico

2.1 Actividad 1 — Diagnóstico: Identificación de problemas en código existente (Python / C++)

Se analiza el siguiente código en Python, el cual presenta múltiples problemas de diseño. El objetivo es identificar los defectos de calidad antes de ejecutarlo.

```
1 # codigo_problematico.py - identifique todos los problemas de diseno
2 class est:
3     def __init__(self, n, a, p):
4         self.n = n
5         self.a = a
6         self.p = p
7         self.h = []
8
9     def add(self, c, n):
10        self.h.append([c, n])
11
12    def ppa(self):
13        if len(self.h) == 0:
14            return 0
15        t = 0
```

```

16         for i in self.h:
17             t = t + i[1]
18         return t / len(self.h)
19
20     def pr(self):
21         print(self.n, self.a, self.p, self.ppa())
22         for i in self.h:
23             print(i)
24
25 e1 = est("Juan Mamani", 20210500, "Ing. Sistemas")
26 e1.add("P00 II", 16)
27 e1.add("BD I", 14)
28 e1.pr()

```

Listing 1: codigo_problematco.py – código con problemas de diseño

Tabla de análisis de calidad

Defecto detectado	Línea(s)	¿Cómo lo corregirías?
Nombre de clase no descriptivo	1	Cambiar <code>est</code> por un nombre descriptivo como <code>Estudiante</code> .
Nombres de atributos y métodos poco descriptivos	2–21	Cambiar <code>n</code> , <code>a</code> , <code>p</code> , <code>h</code> , <code>add</code> , <code>ppa</code> y <code>pr</code> por nombres claros como <code>nombre</code> , <code>codigo</code> , <code>programa</code> , <code>historial</code> , <code>agregar_curso</code> , <code>promedio</code> y <code>mostrar</code> .
Uso de listas con posiciones en lugar de una estructura clara	9, 15–16	En lugar de guardar <code>[c, n]</code> , utilizar una estructura más clara, por ejemplo una clase <code>Curso</code> con atributos <code>nombre</code> y <code>nota</code> .
Método <code>pr()</code> realiza demasiadas tareas	19–21	Separar las responsabilidades: un método para mostrar los datos del estudiante y otro para mostrar los cursos.
Falta de encapsulamiento y validación de datos	2–8	Validar los datos recibidos y controlar el acceso/modificación de los atributos para evitar valores incorrectos.

2.2 Actividad 2 — Refactorización en Python: Misma lógica, mejor diseño (Python / C++)

Se reescribe el código de la Actividad 1 aplicando todas las correcciones identificadas. El resultado debe:

- Tener nombres descriptivos para la clase, atributos y métodos (*snake_case* para Python).
- Usar atributos privados (`_nombre`, `_codigo`) con propiedades `@property` para acceso controlado.
- Separar la lógica de negocio (`calcular_ppa`) de la presentación (`mostrar`).

- Reemplazar listas anidadas [c, n] por un dataclass Nota(curso: str, calificacion: float).
- Validar que la calificación esté en [0, 20] al agregar una nota.

Solución dada (plantilla inicial)

```

1 # SU SOLUCION REFACTORIZADA - Practica S1, Actividad 2
2 from dataclasses import dataclass
3 from typing import Optional
4
5 @dataclass
6 class Nota:
7     curso: str
8     calificacion: float
9
10 class Estudiante:
11     def __init__(self, nombre: str, codigo: int, escuela: str):
12         # TODO: implementar con atributos privados y validacion
13         ...
14
15     @property
16     def nombre(self) -> str:
17         # TODO: getter
18         ...
19
20     def agregar_nota(self, curso: str, calificacion: float) -> None:
21         # TODO: validar [0, 20] y agregar Nota
22         ...
23
24     def calcular_ppa(self) -> float:
25         # TODO: promedio de calificaciones, 0.0 si sin notas
26         ...
27
28     def __str__(self) -> str:
29         # TODO: representacion legible
30         ...
31
32 # Prueba - debe ejecutar sin errores:
33 e = Estudiante(
34     "Juan Carlos Mamani Quispe",
35     20210500,
36     "Ing. Sistemas"
37 )
38 e.agregar_nota("P00 II", 16)
39 e.agregar_nota("Base de Datos I", 14)
40 print(e)
41 print(f"PPA: {e.calcular_ppa():.2f}")
42 try:
43     e.agregar_nota("Redes", 25) # debe lanzar ValueError
44 except ValueError as ex:
45     print(f"Error capturado correctamente: {ex}")

```

Listing 2: Plantilla inicial – Práctica S1, Actividad 2

Solución mejorada

```
1 # Actividad 1 - Codigo refactorizado
2 from dataclasses import dataclass
3
4 @dataclass
5 class Nota:
6     curso: str
7     calificacion: float
8
9 class Estudiante:
10     def __init__(self, nombre: str, codigo: int, escuela: str):
11         self._nombre = nombre
12         self._codigo = codigo
13         self._escuela = escuela
14         self._notas = []
15
16     @property
17     def nombre(self) -> str:
18         return self._nombre
19
20     @property
21     def codigo(self) -> int:
22         return self._codigo
23
24     @property
25     def escuela(self) -> str:
26         return self._escuela
27
28     def agregar_nota(self, curso: str, calificacion: float) -> None:
29         if calificacion < 0 or calificacion > 20:
30             raise ValueError("La calificacion debe estar entre 0 y 20.")
31         self._notas.append(Nota(curso, calificacion))
32
33     def calcular_ppa(self) -> float:
34         if len(self._notas) == 0:
35             return 0.0
36         total = 0
37         for nota in self._notas:
38             total = total + nota.calificacion
39         return total / len(self._notas)
40
41     def mostrar(self) -> None:
42         print(self._nombre, self._codigo, self._escuela)
43         for nota in self._notas:
44             print(nota)
45
46 # Prueba
47 e1 = Estudiante("Juan Mamani", 20210500, "Ing. Sistemas")
48 e1.agregar_nota("POO II", 16)
49 e1.agregar_nota("BD I", 14)
50 e1.mostrar()
51 print("PPA:", e1.calcular_ppa())
```

¿Qué correcciones se aplicaron?

- `est` → `Estudiante`
- `n` → `_nombre`
- `a` → `_codigo`
- `p` → `_escuela`
- `h` → `_notas`
- `add()` → `agregar_nota()`
- `ppa()` → `calcular_ppa()`
- `pr()` → `mostrar()`
- `[c, n]` → `Nota(curso, calificacion)`
- Se agregaron atributos privados (`_nombre`, `_codigo`, `_escuela`, `_notas`).
- Se agregaron propiedades `@property`.
- `calcular_ppa()` quedó separado de `mostrar()`.
- Se agregó validación para que la nota esté entre 0 y 20.

2.3 Actividad 3 — Equivalente en C++17: Encapsulamiento y validación (Python / C++)

Se implementa la clase `Estudiante` en C++17 con encapsulamiento estricto, equivalente a la Actividad 2. Compilación: `g++ -std=c++17 -Wall -o estudiante main.cpp`

Código dado (plantilla inicial)

```
1 // main.cpp - Practica S1, Actividad 3
2 #include <string>
3 #include <vector>
4 #include <stdexcept>
5 #include <numeric>
6 #include <iostream>
7
8 struct Nota {
9     std::string curso;
10    float calificacion;
11 };
12
13 class Estudiante {
14     std::string nombre_;
15     int codigo_;
16     std::string escuela_;
17     std::vector<Nota> notas_;
18 public:
19     Estudiante(std::string nombre, int codigo, std::string escuela) :
20         nombre_(std::move(nombre)),
21         codigo_(codigo),
```

```

22     escuela_(std::move(escuela)) {}
23
24     const std::string& getNombre() const {
25         return nombre_;
26     }
27     int getCodigo() const {
28         return codigo_;
29     }
30
31     void agregarNota(const std::string& curso, float cal) {
32         // TODO: validar 0 <= cal <= 20, lanzar std::invalid_argument si
33         no
34         notas_.push_back({curso, cal});
35     }
36
37     float calcularPPA() const {
38         // TODO: retornar promedio, 0.0f si sin notas
39         if (notas_.empty())
40             return 0.0f;
41         float suma = 0.0f;
42         for (const auto& n : notas_)
43             suma += n.calificacion;
44         return suma / notas_.size();
45     }
46
47     void imprimir() const {
48         std::cout << "["
49             << codigo_
50             << "]" "
51             << nombre_
52             << " ("
53             << escuela_
54             << ")"
55             << " PPA: "
56             << calcularPPA()
57             << '\n';
58     }
59 };
60
61 int main() {
62     Estudiante e(
63         "Juan Carlos Mamani Quispe",
64         20210500,
65         "Ing. Sistemas"
66     );
67     e.agregarNota("POO II", 16);
68     e.agregarNota("Base de Datos I", 14);
69     e.imprimir();
70     try {
71         e.agregarNota("Redes", 25); // debe lanzar
72     } catch (const std::invalid_argument& ex) {
73         std::cout << "Error capturado: "
74             << ex.what()
75             << '\n';

```



```

75     }
76     return 0;
77 }

```

Listing 4: Plantilla inicial – main.cpp, Práctica S1, Actividad 3

Solución a la Actividad 3

```

1  // main.cpp - Practica S1, Actividad 3
2  #include <string>
3  #include <vector>
4  #include <stdexcept>
5  #include <numeric>
6  #include <iostream>
7  #include <utility>
8
9  struct Nota {
10     std::string curso;
11     float calificacion;
12 };
13
14 class Estudiante {
15 private:
16     std::string nombre_;
17     int codigo_;
18     std::string escuela_;
19     std::vector<Nota> notas_;
20 public:
21     Estudiante(std::string nombre, int codigo, std::string escuela)
22         : nombre_(std::move(nombre)),
23           codigo_(codigo),
24           escuela_(std::move(escuela)) {}
25
26     const std::string& getNombre() const {
27         return nombre_;
28     }
29     int getCodigo() const {
30         return codigo_;
31     }
32     const std::string& getEscuela() const {
33         return escuela_;
34     }
35
36     void agregarNota(const std::string& curso, float cal) {
37         if (cal < 0.0f || cal > 20.0f) {
38             throw std::invalid_argument(
39                 "La calificacion debe estar entre 0 y 20."
40             );
41         }
42         notas_.push_back({curso, cal});
43     }
44
45     float calcularPPA() const {
46         if (notas_.empty()) {

```

```

47         return 0.0f;
48     }
49     float suma = 0.0f;
50     for (const auto& n : notas_) {
51         suma += n.calificacion;
52     }
53     return suma / notas_.size();
54 }
55
56 void imprimir() const {
57     std::cout << "["
58               << codigo_
59               << "]" "
60               << nombre_
61               << " ("
62               << escuela_
63               << ")"
64               << " PPA: "
65               << calcularPPA()
66               << '\n';
67 }
68 };
69
70 int main() {
71     Estudiante e(
72         "Juan Carlos Mamani Quispe",
73         20210500,
74         "Ing. Sistemas"
75     );
76     e.agregarNota("P00 II", 16);
77     e.agregarNota("Base de Datos I", 14);
78     e.imprimir();
79     try {
80         e.agregarNota("Redes", 25);
81     }
82     catch (const std::invalid_argument& ex) {
83         std::cout << "Error capturado: "
84                 << ex.what()
85                 << '\n';
86     }
87     return 0;
88 }

```

Listing 5: Actividad 3 – Código completado, main.cpp

2.4 Actividad 4 — Cohesión y acoplamiento: Análisis comparativo (Python / C++)

Se comparan dos diseños y se responden las preguntas de análisis correspondientes.

```

1 # DISEÑO A - todo en una clase
2 class SistemaAcademico:
3     def registrar_estudiante(self, nombre, codigo):

```

```

4         ...
5
6     def agregar_nota(self, codigo_est, curso, nota):
7         ...
8
9     def calcular_ppa(self, codigo_est):
10        ...
11
12    def exportar_csv(self, filename):
13        ...
14
15    def enviar_reporte(self, email):
16        ...
17
18    def conectar_bd(self, host, user, pw):
19        ...

```

Listing 6: Diseño A – todo en una clase

```

1  # DISEÑO B - responsabilidades separadas
2  class Estudiante:
3      def agregar_nota(self, nota: Nota):
4          ...
5
6      def calcular_ppa(self) -> float:
7          ...
8
9  class RepositorioEstudiante:
10     def guardar(self, e: Estudiante):
11         ...
12
13     def buscar(self, codigo: int) -> Estudiante:
14         ...
15
16 class ExportadorCSV:
17     def exportar(
18         self,
19         estudiantes: list[Estudiante],
20         archivo: str
21     ):
22         ...

```

Listing 7: Diseño B – responsabilidades separadas

Preguntas de análisis

Pregunta de análisis	Su respuesta
¿Cuál diseño tiene mayor cohesión? ¿Por qué?	El Diseño B, porque cada clase tiene una responsabilidad específica y relacionada.

Pregunta de análisis	Su respuesta
¿Qué ocurre con el Diseño A si cambia el formato de exportación?	Se tendría que modificar la clase <code>SistemaAcademico</code> , aunque el cambio solo corresponda a la exportación.
¿Qué principio SOLID viola el Diseño A? (lo verán formalmente en la Semana 7)	Viola el Principio de Responsabilidad Única (SRP), porque una sola clase tiene muchas responsabilidades.
¿El Diseño B es más fácil de probar unitariamente? ¿Por qué?	Sí, porque cada clase tiene una responsabilidad independiente y se puede probar por separado.

3. Conclusiones

- El análisis del código original permitió comprobar que un programa puede ejecutarse correctamente y, aun así, presentar defectos de diseño graves, como nombres poco descriptivos, ausencia de encapsulamiento y estructuras de datos poco expresivas (listas posicionales en lugar de objetos), lo que dificulta su lectura, mantenimiento y escalabilidad.
- La refactorización realizada en Python demostró que aplicar convenciones de nomenclatura clara, atributos privados con propiedades `@property` y `dataclasses` mejora significativamente la legibilidad del código sin alterar su comportamiento funcional, lo cual es el objetivo central de toda refactorización.
- La validación de datos en el método `agregar_nota`, tanto en Python (mediante `ValueError`) como en C++17 (mediante `std::invalid_argument`), evidencia la importancia de proteger la integridad de los objetos desde el momento en que se construyen o modifican, evitando estados inconsistentes.
- La implementación equivalente en C++17 permitió reforzar que los principios de encapsulamiento son independientes del lenguaje: aunque la sintaxis cambie (uso de `private`, referencias constantes, excepciones estándar), la lógica de diseño –ocultar el estado interno y exponerlo solo a través de una interfaz controlada– se mantiene igual en Python y en C++.
- El análisis comparativo de la Actividad 4 permitió identificar de forma práctica los conceptos de cohesión y acoplamiento: un diseño donde una sola clase concentra múltiples responsabilidades (persistencia, exportación, envío de reportes, conexión a base de datos) resulta más frágil ante cambios, mientras que separar dichas responsabilidades en clases independientes facilita las pruebas unitarias y reduce el impacto de futuras modificaciones.
- En conjunto, la práctica evidencia que el Principio de Responsabilidad Única (SRP), aunque se formalizará en semanas posteriores del curso, ya puede reconocerse de manera intuitiva al observar cómo un diseño mal cohesionado obliga a modificar una misma clase por razones no relacionadas entre sí, lo cual constituye una señal de alerta en el

diseño de software orientado a objetos.

4. Referencias

- Booch, G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J., & Houston, K. A. (2007). *Object-oriented analysis and design with applications* (3.^a ed.). Addison-Wesley.
- Lutz, M. (2013). *Learning Python* (5.^a ed.). O'Reilly Media.
- Martin, R. C. (2000). *Design principles and design patterns*. Object Mentor.
- Meyers, S. (2014). *Effective modern C++: 42 specific ways to improve your use of C++11 and C++14*. O'Reilly Media.
- Python Software Foundation. (2024). *dataclasses — Data classes*. Python Documentation. <https://docs.python.org/3/library/dataclasses.html>
- Stroustrup, B. (2018). *A tour of C++* (2.^a ed.). Addison-Wesley.